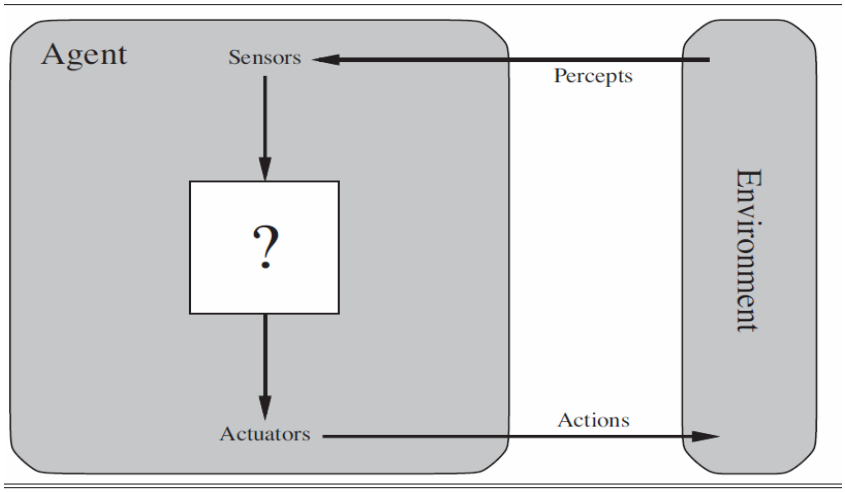
Agents

What is an Agent?

An **agent** is anything that can be viewed as perceiving its **environment** through **sensors** and acting upon that environment through **actuators**. This simple idea is illustrated in Figure below.



Agents interact with environments through sensors and actuators.

An agent can be:

- **Human-Agent:** A human agent has eyes, ears, and other organs which work for sensors and hand, legs, vocal tract and so on for actuators.
- **Robotic Agent:** A robotic agent can have cameras, infrared range finder for sensors and various motors for actuators.
- **Software Agent:** A software agent receives keystrokes, file contents, and network packets as sensory inputs and acts on the environment by displaying on the screen, writing files, and sending network packets.

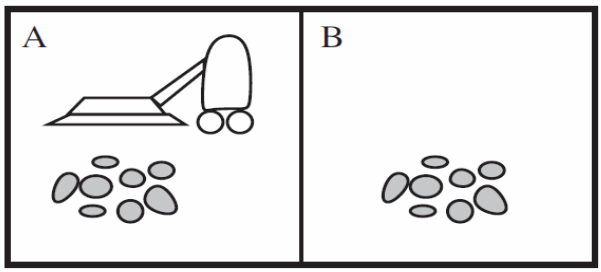
An Agent runs in the cycle of **perceiving**, **thinking**, and **acting**.

Some terms to know:

- **Perception**: what agent see in the environment thru sensors(sense organs).
  Sensor: Sensor is a device which detects the change in the environment and sends the information to other electronic devices. A robotic/autonomous agent observes its environment through sensors.
- **Perception history**: It is the history of agent perception which comes in a specific period.
- **Actuators:** an actuator is the actual mechanism that enables the effector to execute an action, component of machines that converts energy into motion. The actuators are only responsible for moving and controlling a system. An actuator can be an electric motor, gears, rails, etc.
- **Effectors:** Effectors are the Agents organs (hands and legs)/ a device that affects physical environment. Effectors can be legs, wheels, arms, fingers, wings, fins, and display screen.
- **agent function** that maps any given percept sequence to an action, Agent's behavior is *mathematically* described by **Agent function**

- The agent function for an artificial agent will be implemented practically by an **agent program**.

example—the vacuum-cleaner world shown in Figure below.



A vacuum-cleaner world with just two locations.

This particular world has just two locations: squares A and B.
One very simple **agent function** is the following:

- Percepts: location/which square it is in and contents, e.g., [A,Dirty]
- Actions:Move *Left*, *Right*, *Suck up dirt*, *NoOp*
- *Agent's function* → *look-up table,* if the current square is dirty, then suck; otherwise, move to the other square.*For many agents this is a very large table*

Percept sequence	Action
$[A, Clean]$	$Right$
$[A, Dirty]$	$Suck$
$[B, Clean]$	$Left$
$[B, Dirty]$	$Suck$
$[A, Clean], [A, Clean]$	$Right$
$[A, Clean], [A, Dirty]$	$Suck$
⋮	⋮
$[A, Clean], [A, Clean], [A, Clean]$	$Right$
$[A, Clean], [A, Clean], [A, Dirty]$	$Suck$
⋮	⋮

3 Partial tabulation of a simple agent function for the vacuum-cleaner world

- An **agent program** for this agent is shown in Figure below

```
function REFLEX-VACUUM-AGENT([location,status]) returns an action
  if status = Dirty then return Suck
  else if location = A then return Right
  else if location = B then return Left
```

GOOD BEHAVIOR: THE CONCEPT OF RATIONALITY

Rational Agent

A rational agent is one that does/ performs the right things.

For an AI agent, the rational action is most important because in AI reinforcement learning algorithm, for each best possible action, agent gets the positive reward and for each wrong action, an agent gets a negative reward.AI is about creating rational agents to use for game theory and decision theory for various real-world scenarios.

Rationality

The rationality of an agent is measured by its **performance measure** that evaluates any given **sequence of environment states** and **action sequences**, if sequence leads to **desirable state** then it performed well.

No universal performance measure for all tasks and agents. Consider, for example, the vacuum-cleaner agent we might propose to measure performance by

- the amount of dirt cleaned up in a single eight-hour shift.
- A rational agent can maximize this performance measure by cleaning up the dirt, then dumping it all on the floor, then cleaning it up again, and so on.
- A more suitable performance measure would reward the agent for having a clean floor. For example, one point could be awarded for each clean square at each time step (perhaps with a penalty for electricity consumed and noise generated).

As a **GENERAL RULE**, it is better to design performance measures according to w**hat one actually wants in the environment,** rather than according to how one thinks the agent should behave. E.g., in vacuum-cleaner world, We want the floor clean, no matter how the agent behave

- performance measure of a vacuum-cleaner agent could be amount of dirt cleaned up, amount of time taken, amount of electricity consumed, amount of noise generated, etc.

What is rational at any given time depends on four things

- **Performance measure that defines the success criterion.**
- **Agent prior knowledge of its environment.**
- **Best possible actions that an agent can perform.**
- **The agent's percept sequence to date**

This leads to a **definition of a rational agent=>For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.***E.g., an exam, maximize marks, based on the questions on the paper & your knowledge*

Omniscience, learning, and autonomy

We need to be careful to distinguish between rationality and omniscience. An omniscient agent knows the actual outcome of its actions and can act accordingly; but omniscience is impossible in reality.

Example:

- You're walking on the Champs Élysées and see your friend across the street. There's no traffic, and it's safe, so you start to cross.
- Suddenly, a cargo door from a plane falls and hits you.
- Question: Were you irrational?
- Answer: No! You made the best decision based on what you knew.

Concept	Explanation
Rationality	Doing the best you can with the **information available**.
Perfection	Doing what turns out to be best **after everything happens**.

- A rational agent doesn't have to know everything (omniscience).
  It just needs to make the best decision based on what it has seen (its percepts) so far
- **Example:** Crossing the Road Imagine an agent (like a robot or person):
- It doesn't look both ways before crossing.
- So it doesn't know a truck is coming.
- Should it still cross because its "view" says nothing is wrong?
- No! That would still be irrational.

Why?

- Risk is high — Crossing without checking is dangerous.
- A rational agent should first look (gather info) to improve its future decisions.
- This action of gathering useful info is called information gathering.

Another Example: Vacuum Cleaner

In an unknown house, a vacuum agent: Should explore rooms first (gather info). So it knows where dirt is and what path to follow

Rational agents don't just act—they also collect info first to make smarter decisions.

Rational Agents Must Learn from Experience

A good agent doesn't just act—it also:

- Gathers information
- Learns from what it sees
- Updates its behavior if things change
  - An autonomous agent is one that makes decisions based on its own observations and experiences, rather than just relying on instructions given by its designer.

- A rational agent should be autonomous so it can learn from the environment and adapt when the initial knowledge it was given is incomplete or incorrect.
- For example, a vacuum-cleaning agent that learns where dirt is likely to appear will perform better than one that blindly follows fixed rules.
- However, full autonomy is not needed right from the beginning—an agent with no experience would act randomly without some basic built-in knowledge, just like animals are born with instincts to help them survive until they learn.

THE NATURE OF ENVIRONMENTS

Task environments, which are the "problems" to which rational agents are the "solutions."

Specifying the task environment

PEAS Representation

PEAS is a type of model on which an AI agent works upon. When we define an AI agent or rational agent, then we can group its properties under PEAS representation model. It is made up of four words:

- **P:** Performance measure
- **E:** Environment
- **A:** Actuators
- **S:** Sensors

Here performance measure is the objective for the success of an agent's behavior.

PEAS for vacuum cleaner	
Performance Measure	cleanness, efficiency: distance traveled to clean, battery life, security
Environment	room, table, wood floor, carpet, different obstacles
Actuators	wheels, different brushes, vacuum extractor
Sensors	camera, dirt detection sensor, cliff sensor, bump sensors, infrared wall sensors

PEAS for self-driving cars:

- Performance measure
  - How can we judge the automated driver?
  - Which factors are considered?
    - getting to the correct destination
    - minimizing fuel consumption
    - minimizing the trip time and/or cost
    - minimizing the violations of traffic laws
    - maximizing the safety and comfort, etc.

Environment

- A taxi must deal with a variety of roads
- Traffic lights, other vehicles, pedestrians, stray animals, road works, police cars, etc.

- Interact with the customer
- Actuators (for outputs)
  - Control over the accelerator, steering, gear shifting and braking
  - A display to communicate with the customers
- Sensors (for inputs)
  - Detect other vehicles, road situations
  - GPS (Global Positioning System) to know where the taxi is
  - Many more devices are necessary

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits	Roads, other traffic, pedestrians, customers	Steering, accelerator, brake, signal, horn, display	Cameras, sonar, speedometer, GPS, odometer, accelerometer, engine sensors, keyboard

Figure 2.4 PEAS description of the task environment for an automated taxi.

Example of Agents with their PEAS representation

Agent Type	Performance Measure	Environment	Actuators	Sensors
Medical diagnosis system	Healthy patient, reduced costs	Patient, hospital, staff	Display of questions, tests, diagnoses, treatments, referrals	Keyboard entry of symptoms, findings, patient's answers
Satellite image analysis system	Correct image categorization	Downlink from orbiting satellite	Display of scene categorization	Color pixel arrays
Part-picking robot	Percentage of parts in correct bins	Conveyor belt with parts; bins	Jointed arm and hand	Camera, joint angle sensors
Refinery controller	Purity, yield, safety	Refinery, operators	Valves, pumps, heaters, displays	Temperature, pressure, chemical sensors
Interactive English tutor	Student's score on test	Set of students, testing agency	Display of exercises, suggestions, corrections	Keyboard entry

Figure 2.5 Examples of agent types and their PEAS descriptions.

Environment:

An environment is **everything in the world which surrounds the agent**, but it is not a part of an agent itself. An environment can be described as a situation in which an agent is present.

Properties of task environments

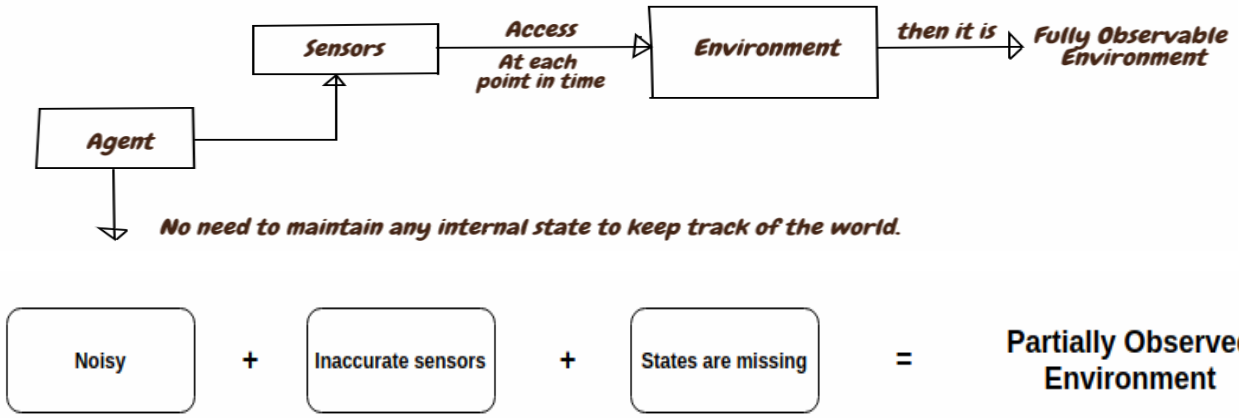
As per Russell and Norvig, an environment can have various features from the point of view of an agent:

1. Fully observable vs Partially Observable
2. Static vs Dynamic
3. Discrete vs Continuous
4. Deterministic vs Stochastic
5. Single-agent vs Multi-agent
6. Episodic vs sequential
7. Known vs Unknown

For example, program a chess bot, the environment is a chessboard and creating a room cleaner robot, the environment is Room.

Fully observable vs Partially Observable:

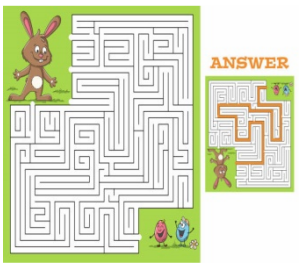
- If an agent sensor can sense or access the complete state of an environment at each point of time then task is **a fully observable** environment, else it is **partially observable**.
- A fully observable environment is easy as there is no need to maintain the internal state to keep track history of the world. Eg. : Sudoku, 8-puzzle problem : The state is completely visible at any point of time.
- **Chess –** the board is fully observable, so are the opponent's moves.
- Eg. : Automated Car Driving System : You cannot predict what other car drivers do, and how they are going to drive or what decision they are going to make while driving.-Partially observable



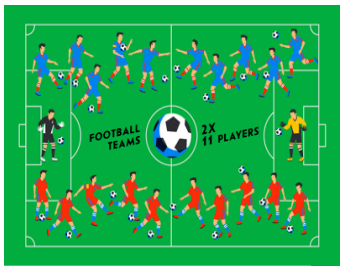
- An environment is called **unobservable** when the agent has no sensors in all environments.

Single-agent vs Multi-agent

- If only one agent is participated/involved in an environment, and operating by itself then such an environment is called single agent environment.
- **Real-life Example**: Playing tennis against the ball is a single agent environment where there is only one player.
- A person left alone in a maze is an example of the single-agent system. A maze is **a type of puzzle games where a player moves in complex and branched passages to find a particular target or location**.
- However, if More than one agent is taking actions in the environment then such an environment is called a multi-agent environment.



- **Real-life Example**: The game of football is multi-agent as it involves 11 players in each team.

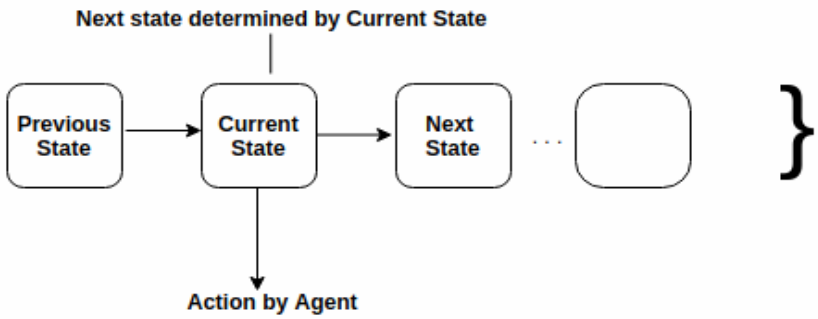


Further, there are two types of "Multi Agent" environments, **Competitive and Collaborative**

In competitive Al environments various Al agents are made to **work against** each other with an intent to maximize the outcome. This means that the agents compete with each other to move towards the goal EX, agents compete against each other in order to win the **game of chess**. Whereas, in collaborative AI environments different Al agents **work together** in order to achieve a goal, ex **automated cars, or smart home sensors** work in collaborative Al environments as they work with each other in order to avoid collisions and reach their desired destinations.

Deterministic vs Stochastic:

- When the **next state of the environment is completely determined by the current state & the action executed by the agent,** then we say the environment is deterministic, otherwise it is stochastic.

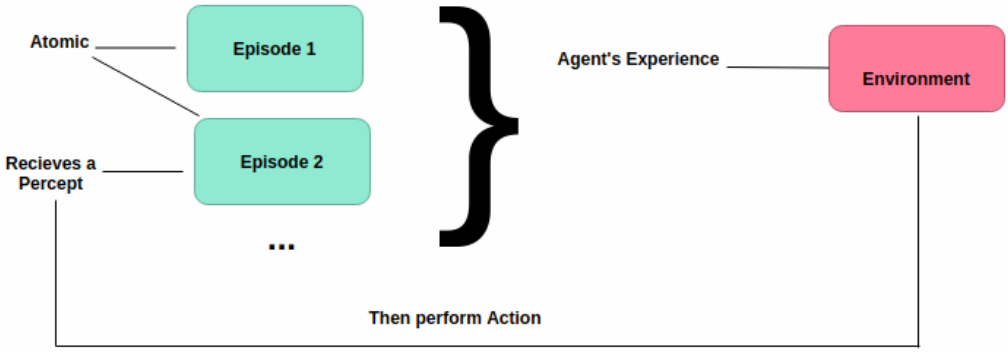


- **Chess –** there would be only a few possible moves for a coin at the current state and these moves can be determined.
- A stochastic environment is random in nature and cannot be determined completely by an agent.
- **Self-Driving Cars –** the actions of a self-driving car are not unique, it varies time to time.

Episodic vs Sequential

- An episode = agent's single pair of perception & action
- In an "Episodic" environment, agent's actions are divided into atomic episodes.

- In each episode, the agent receives a percept and performs a single action. The current episode is different from the previous one and there is no dependency between them. Every episode is independent of each other, Episodic environment is simpler.
- The agent does not need to think ahead.
- **Example:** Consider an example of **Pick and Place robot**, which is used to detect defective parts from conveyer belt. Here, every time robot(agent) will make decision on current part i.e. there is no dependency between current and previous decision.



- **The sequential** environment is where the **next state is dependent on the current action.** So agent current action can change all of the future states of the environment. Here, an agent requires memory of past actions to determine the next best actions.
- Eg. : **Automatic car driving:** If the driver wants to apply break, the next actions include press clutch and then lower the gear.
- **Ex: Checkers-** Where previous move can affect all the following moves.

Static vs Dynamic

- The environment is referred to as "Static", if the **environment remains unchanged while the agent is performing some task/percepting. Static** environments are easy to handle because the agent need not keep looking at the world while deciding on an action, nor need it worry about the passage of time.
- Eg. : Sudoku cross-word puzzle, Vacuum cleaner world : There is no change in the environment when an agent plays sudoku or cleans the dirt, room is static while cleaning, in respective examples.

- **Dynamic** Environment could be changed while an agent is percepting the environment. So agents keep looking at the environment while taking each action.
- Eg : Automatic car driver : The car driver's movement will cause the other car's to make adjustments, thus causing a change in the environment.



- A roller coaster ride is dynamic as it is set in motion and the environment keeps changing every instant.
- Playing soccer is a dynamic environment where players' positions keep changing throughout the game. So a player hit the ball by observing the opposite team.

- A “Semi-Dynamic” environment is the one where environment remains unchanged but the agent’s performance changes.
- **Real-life Example**: chess when played with a clock is semi-dynamic,

Discrete vs Continuous

- The types of environments in AI in which we have a **distinct, limited/finite states** and clearly defined input (percepts) and outputs (actions) is called as “Discrete” environment.
- **Real-life Example**: The game of chess is discrete as it has only a finite number of moves. The number of moves might vary with every game, but still, it’s finite.
- While in a **Continuous environment**, the environment can have **an infinite number of states.** So the possibilities of taking an action are also infinite cannot be defined beforehand and agent receives a continuous input.
- Eg. : Vision systems in drones or self-driving cars operate on continuous AI environments as their actions are driving, parking, etc. which cannot be finite, env keeps on changing.
- **Real-life Example**: In a basketball game, the position of players (**Environment**) keeps changing continuously and hitting (**Action**) the ball towards the basket can have different angles and speed so infinite possibilities.

Known vs Unknown env

- Known and unknown are not actually a feature of an environment, but it is an agent's state of knowledge to perform an action.
- In a known environment, the results/outcomes/**output for all actions are known**/given to the agent. (example: solitaire card games).
- If the environment is unknown, the agent will **have to learn how it works and has to gain knowledge about how the environment** works for better decision making to perform an action.(example: new video game).

These environments are the good examples of **Exploitation** (Known Environment) and **Exploration** (Unknown Environment) which come in **Reinforcement Learning**.

- There are mainly six groups of environment and an environment can be in multiple groups. Below are more real-life examples and categories into environment groups.

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Interactive English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

Figure 2.6 Examples of task environments and their characteristics.

THE STRUCTURE OF AGENTS

Operations of an Agent: Agent Program, Agent Function are the operations.

The task of AI is to design an agent program which implements the agent function.

The structure of an intelligent agent is a combination of architecture and agent program. It can be viewed as: **Agent = Architecture + Agent program**

Architecture: Architecture is machinery that an AI agent executes on.

Agent Function: Agent function is used to map a percept to an action.

$$f: P^* \rightarrow A$$

Agent program: Agent program is an implementation of agent function. An agent program executes on the physical architecture to produce function f. It runs on some sort of computing device with physical sensors and actuators — call this as the Architecture.

Table-lookup agent

• A trivial agent program: keeps track of the percept sequence and then uses it to index into a table of actions to decide what to do

• The designers must construct the table that contains the appropriate action for every possible percept sequence.

```
function TABLE-DRIVEN-AGENT(percept) returns an action
    static:   percepts, a sequence, initially empty
              table, a table of actions, indexed by percept sequences, initially fully specified
    append percept to the end of percepts
    action <--LOOKUP(percepts, table)
    return action
```

Drawbacks:

– Huge table (P^T, **P**: set of possible percepts, **T**: lifetime)

- •Space to store the table
- •Take a long time to build the table
- •No autonomy
- •Even with learning, need a long time to learn the table entries

Rather than a table how we can produce rational behavior from a small amount of code =>

Categorized into five types in **order of increasing generality and based on their degree of perceived intelligence and capability.** All these agents can improve their performance and generate better action over the time. These are given below:

- Simple Reflex Agent
- Model-based reflex agent
- Goal-based agents
- Utility-based agent

- Learning agent

1. Simple Reflex agent:

- The simplest kind of agent is the **simple reflex agent**. These agents **select actions on the basis of the *current* percept, igoring the rest of the percept history.**
- w/o any calculation immediately/spontaneously take/perform an action
- For example, the vacuum agent whose agent function is tabulated is a simple reflex agent, because its decision is based only on the current location and on whether that location contains dirt.

• Example: simple reflex vacuum cleaner agent

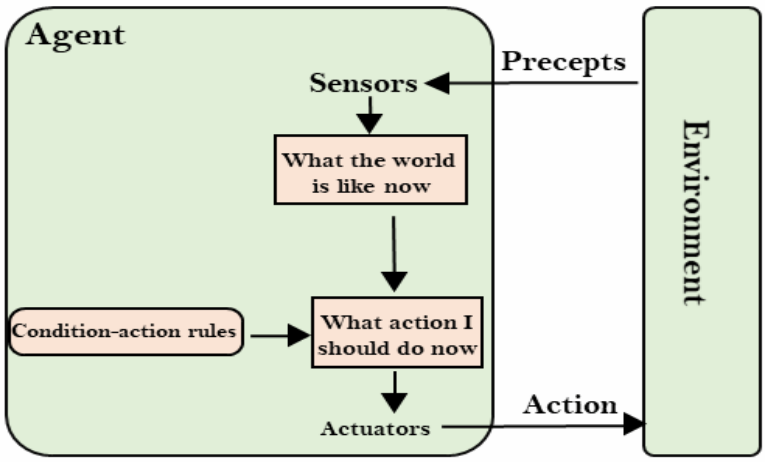
```
function REFLEX-VACUUM-AGENT([location,status]) returns an action
    if status = Dirty then return Suck
    else if location = A then return Right
    else if location = B then return Left
```

The Simple reflex agent works with agent function as Condition-action rule.

A condition-action rule is a rule that maps a state i.e, condition/current sate to an action. If the condition is true, then the action is taken, else not.

Such as a Room Cleaner agent, it works only if there is dirt in the room.

"The car in front is braking." Then, this triggers some established connection in the agent program to the action "initiate braking." We call such a connection a **condition–action rule**, written as **if** *car-in-front-is-braking* **then** *initiate-braking*.



Above gives the structure of this general program in schematic form, **Depicts how condition-action rule allow the Simple reflex agent to make the connection from percept to action**

The program above is specific to one particular vacuum environment. A more general and flexible approach is first to **build a general-purpose interpreter** for **condition– action** rules and **then to create rule sets** for specific task environments.

```
function SIMPLE-REFLEX-AGENT(percept) returns an action
  persistent: rules, a set of condition–action rules

  state ← INTERPRET-INPUT(percept)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

Figure 2.10 A simple reflex agent. It acts according to a rule whose condition matches the current state, as defined by the percept.

The INTERPRET-INPUT function generates an abstracted description of the current state from the percept, and

the RULE-MATCH function returns the first rule in the set of rules that matches the given state description.

Put else part also for completeness

- Problems for the simple reflex agent design approach:
  - Simple-reflex agents are **simple,** but they turn out to be of **very limited intelligence.**
  - The agent will work only if the **correct decision** can be made on the basis of the **current percept** – that is only if the **environment is fully observable**.
  - If there occurs any change in the environment, then the collection of rules need to be updated.
  - **Infinite loops** are often unavoidable when agents operating in partially observable environments – escape could be possible if the agent can randomize its actions.

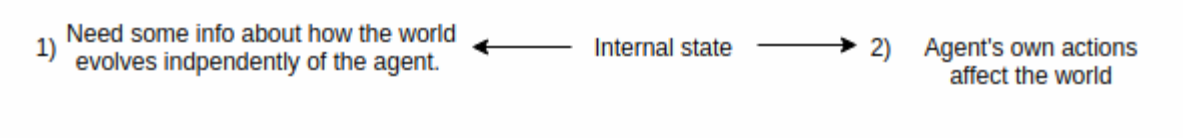
2. Model-based reflex agent – agent with memory

- For the world that is partially observable, the agent should keep track of the part of the world it can't see now
  - That depends on the percept history
  - Reflecting some of the unobserved aspects of current state
  - E.g., driving a car and changing lane

- The agent has to keep track of the internal state which is adjusted by each percept and that depends on the percept history. The current state is stored inside the agent which maintains some kind of structure describing the part of the world which cannot be seen.
- So, **Current State = Old Internal State + Current Percept**
- **Update Internal state** : Updating the internal state information as time goes by requires two kinds of knowledge to be encoded in the agent program

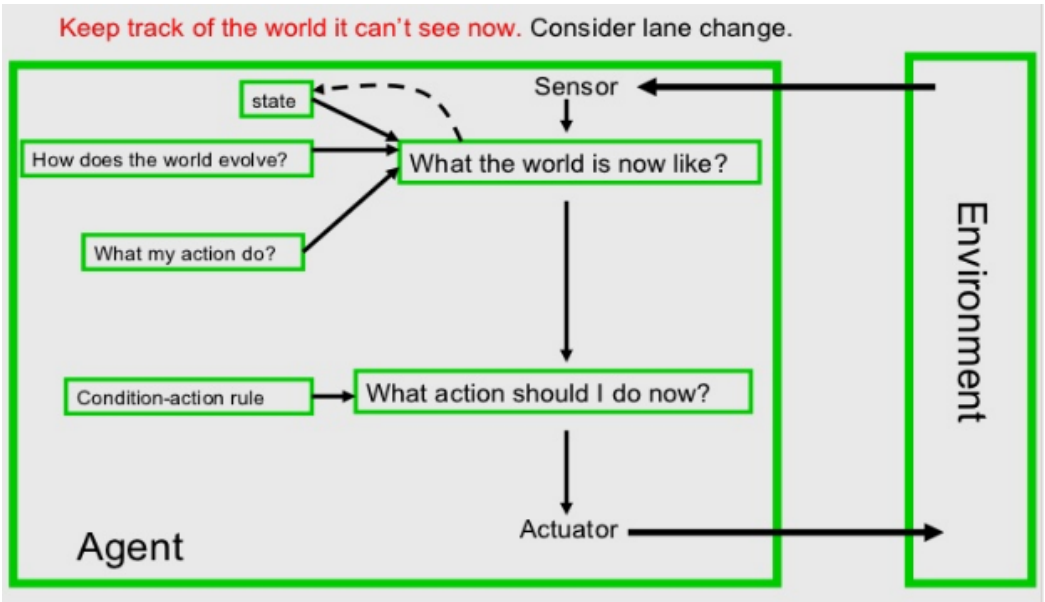
  – Information about how the world evolves independently of the agent

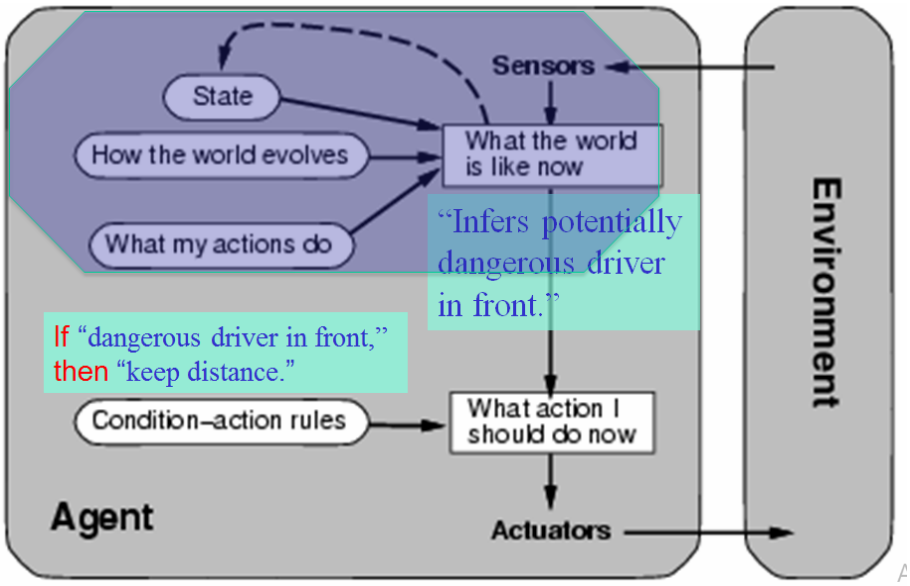
  – Information about how the agent's own actions affects the world



- A model-based agent has two important factors:
  - **Model:** it is the knowledge "about How the world works" Whether implemented in simple boolean circuits or in complete scientific theories is called the Model of the World. An agent that uses such a model is called a **Model-Based-Agent**.
  - **Internal State:** It is a representation of the current state based on percept history.
- These agents have the model, "which is knowledge of the world" and based on the model they perform actions.

Structure of the Model-based Reflex Agent





```
function MODEL-BASED-REFLEX-AGENT(percept) returns an action
  persistent: state, the agent's current conception of the world state
              model, a description of how the next state depends on current state and action
              rules, a set of condition–action rules
              action, the most recent action, initially none

  state ← UPDATE-STATE(state, action, percept, model)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

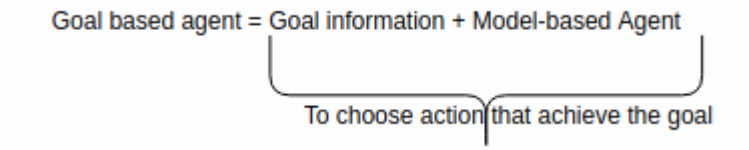
Figure 2.12 A model-based reflex agent. It keeps track of the current state of the world, using an internal model. It then chooses an action in the same way as the reflex agent.

3. Goal-based agents

- In life, in order to get things done we set goals for us to achieve, this pushes us to make the right decisions when we need to.

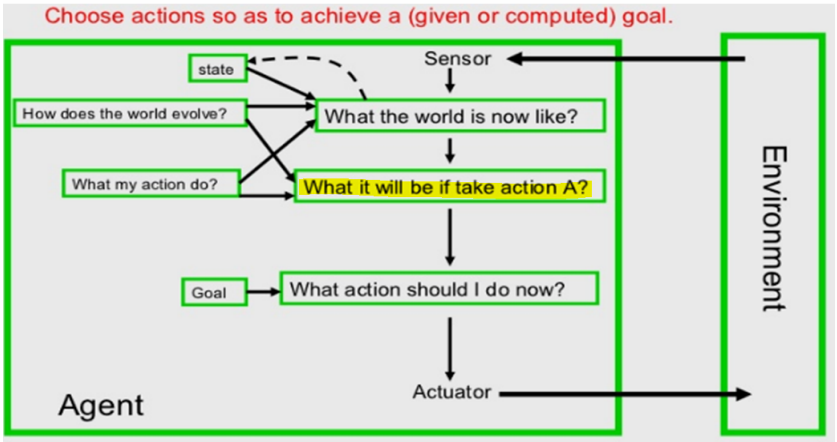
- A simple example would be the shopping list; our goal is to pick up every thing on that list. This makes it easier to decide if you need to choose between milk and orange juice because you can only afford one. As milk is a goal on our shopping list and the orange juice is not we chose the milk.

- **The knowledge of the current state environment is not always sufficient to decide for an agent to what to do.** (e.g. decision at a road junction)
- Goal-based agents expand the capabilities of the model-based agent by having the "goal" information.



- They choose an action, so that they can achieve the goal.
- These kinds of agents take decisions based on how far they are currently from their **goal**(description of desirable situations). Their every action is intended to reduce its distance from the goal.
- So in an intelligent agent having a set of goals with desirable situations are needed.
- The agent program can use these goal information along with **set of possible actions** and their **predicted outcomes** in order to choose which action(s) achieve our goal(s).
- The knowledge that supports its decisions is represented explicitly and can be modified, which makes these agents more flexible. They usually require search and planning, which makes an agent proactive.. The goal-based agent's behavior can easily be changed.

- Unlike the previous reflex agents before just acting when condition matches, this agent reviews many/ a long sequence of possible actions and c**hooses the one which come closest to achieving its goals,** whereas the reflex agents just have an automated response for certain situations.



4. Utility-based agents

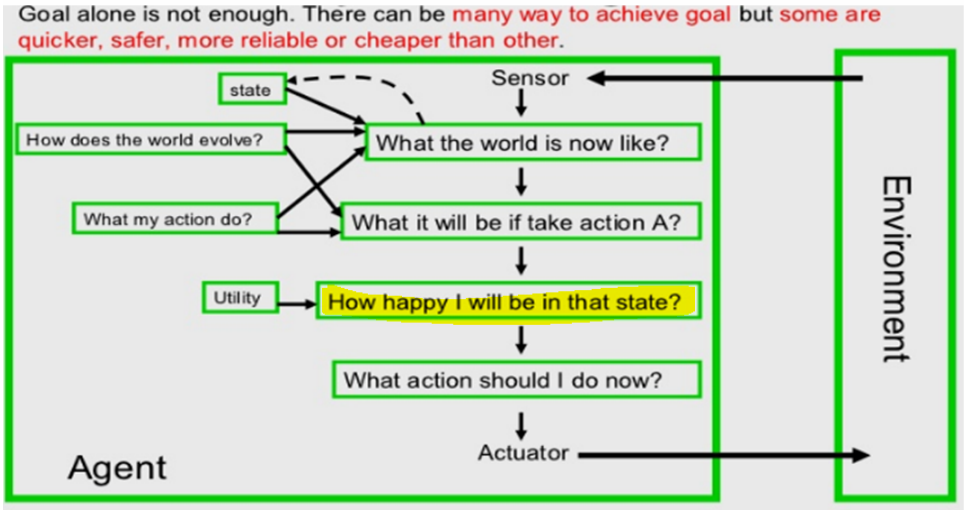
Sometimes achieving the desired goal is not enough. We may look for a **quicker, safer, cheaper trip** to reach a destination.

Agent happiness should be taken into consideration- they just provide a binary distinction between happy and unhappy states. E.g. meals in Canteen, good or not ?

Many action sequences -> the goals

- some are better and some worse
- If goal means success, then **utility** means the degree of success (how happy agent is if reached a goal state) =>driving car=>utility=> faster/ safer

- Utility describes how **"happy"** the agent is.
- Utility-based agent act based on not only goals but also the best way to achieve the goal.
  - These agents are similar to the goal-based agent but provide an extra component of utility measurement which makes them different by providing a **measure of success at a given state.**
- When there are multiple possible alternatives, then to decide which action is best, utility-based agents are used. They choose actions based on a **preference (utility)** for each state
  - The utility function maps each state to a real number which **describes the associated degree of happiness.**



  - A rational utility-based agent chooses the action that maximizes the expected utility of the actions outcomes.

5. Learning Agents

- After an agent is programmed, can it work immediately?
  - No, it still need teaching
- Turing **– instead of actually programming intelligent machines by hand, which is too much work, build learning machines and then teach them**

- In AI,
  - Once an agent is done
  - We teach it by giving it a set of examples
  - Test it by using another set of examples
- Based on performance of agent on unknown examples, We then say the agent learns
  - A learning agent

- A learning agent in AI is the type of agent which can learn from its past experiences, or it has learning capabilities.
- Learning agent starts to act with basic knowledge in initially unknown environments and then able to act and adapt automatically through learning and become more competent than its initial knowledge alone might allow.
- A learning agent has mainly four conceptual components, which are:

i. **Learning element:** It is responsible for making improvements by learning from environment

ii. **Performance element:** It is responsible for selecting an external action- it takes in percepts and decides on actions.

iii. **Critic:** The learning element takes feedback from critics which describes how well the agent is doing with respect to a fixed performance standard and determines how the performance element should be modified to do better in the future.

(Feedback from user or examples, good or not?)

iv. **Problem generator:** This component is responsible for suggesting actions that will lead to new and informative experiences.

Automated taxi example:

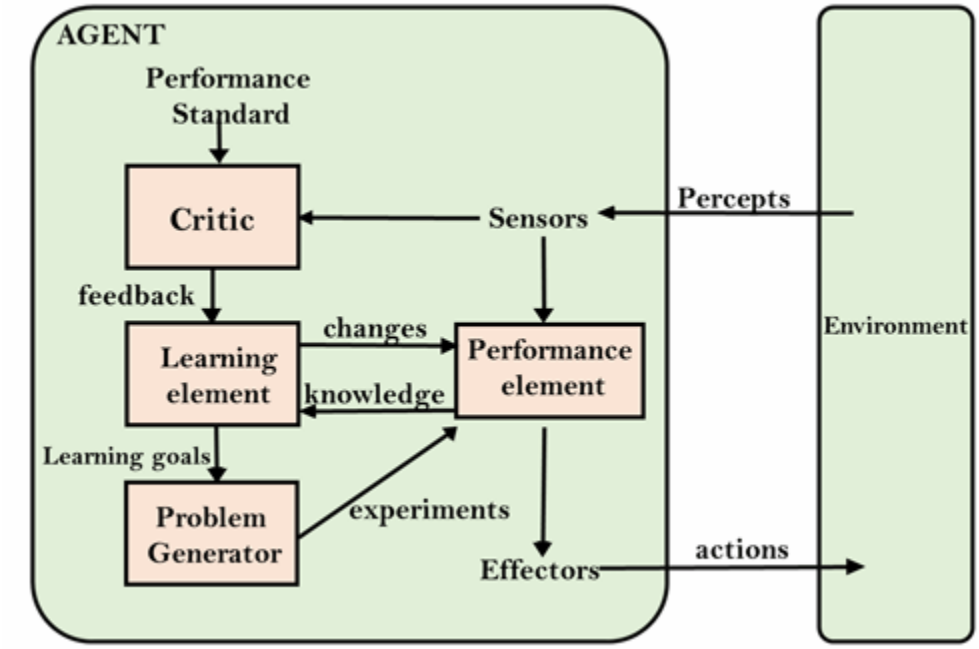
The performance element consists of whatever collection of knowledge and procedures the taxi has for **selecting its driving actions**. The taxi **goes out** on the road and drives, using this **performance element**. The critic **observes the world** and **passes information** along to the learning element.

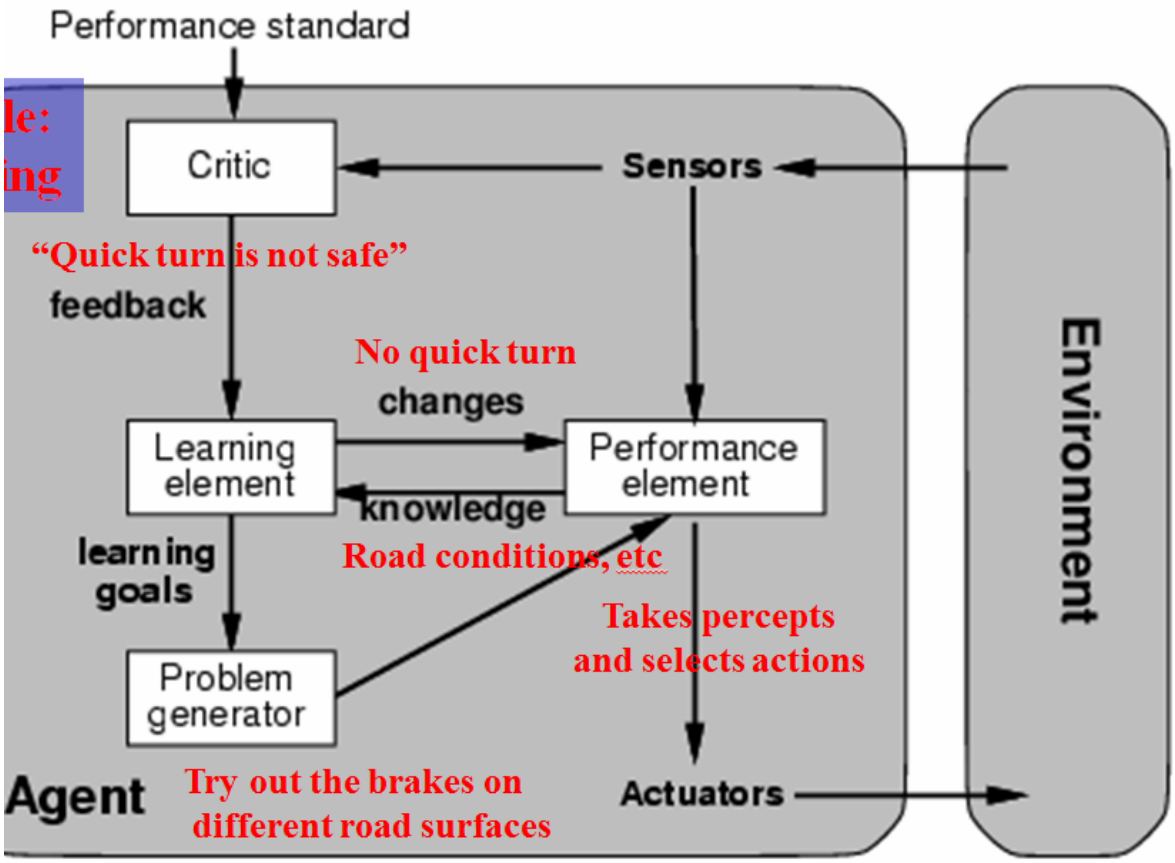
For example, after the taxi makes a quick left turn across three lanes of traffic, the **critic** observes the shocking language used by other drivers.

From this experience, the **learning element** is able to formulate a rule saying this was a bad action, and the **performance element is modified** by installation of the new rule.

The **problem generator** might identify certain areas of behavior in need of improvement and suggest experiments, such as trying out the brakes on different road surfaces under different conditions.

Hence, learning agents are able to learn, analyze performance, and look for new ways to improve the performance.


AGENT
Performance Standard
Critic
Sensors
Percepts
feedback
changes
Learning element
Performance element
knowledge
Environment
Learning goals
Problem Generator
experiments
Effectors
actions


Performance standard
Critic
Sensors
"Quick turn is not safe"
feedback
No quick turn
changes
Learning element
Performance element
knowledge
learning goals
Road conditions, etc
Takes percepts and selects actions
Problem generator
Environment
Agent
Try out the brakes on different road surfaces
Actuators

Steps performed by Problem-solving agent – formulate, search, execute

1. **Goal Formulation**
2. **Problem formulation**

   Components in problem definition

   a. **Initial State**
   b. **Actions**
   c. **Transition Model**
   d. **Goal Test**
   e. **Path cost**

3. **Search** for solution: Given the problem, search for a solution --- *a sequence of actions to achieve the goal starting from the initial state.=>explore the state space*
4. **Execution of the solution**

- **Goal Formulation:** It is the first and simplest step in problem-solving. It organizes the steps required to formulate one goal out of multiple goals and which requires some actions to achieve that goal. Goal formulation is based on the current situation and the agent's performance measure.
- **Problem formulation:** process of deciding what actions and states to consider, given a goal.

For now, let us assume that the agent will consider actions at the level of driving from one major town to another.

Each state therefore corresponds to being in a particular town.

Problem: On holiday in country Romania currently in Arad. Flight leaves tomorrow to Bucharest. Find a short route to drive to Bucharest.

Formulate problem:

states: various cities

actions: drive between cities

solution: sequence of cities, e.g., Arad, Sibiu, Fagaras, Bucharest

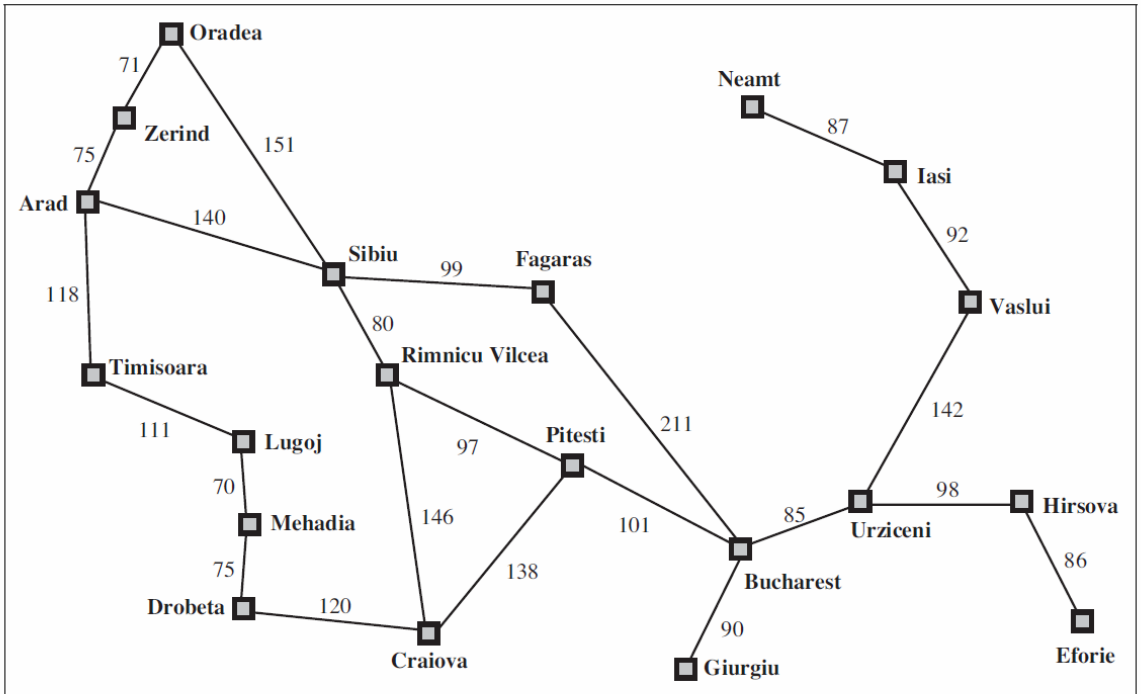


Figure 3.2 A simplified road map of part of Romania.

Properties of the basic search Agent

Under the following assumptions about the environment, the solution to any problem is a fixed sequence of actions.

1. **Observable=>** agent always **knows the current state.**
   supposing that each city on the map has a sign indicating its presence to arriving drivers.
2. **Discrete => only finite actions to choose from.**
   each city is connected to a small number of other cities.
3. **Known =>** agent knows **which states are reached by each action.**
4. **Deterministic => each action has exactly one outcome.**

After **formulating a goal and a problem to solve**, the agent calls a **search** procedure to **solve it.** It then **uses the solution** to guide its **actions**, doing whatever the solution recommends as the next thing to do.

- **Search:** The process of **looking for a sequence of actions that reaches the goal i**s called search.
- **Solution:** A search algorithm takes a **problem as input** and **returns a solution** in the form of an **action sequence**. It finds the best algorithm out of various algorithms, which may be proven as the best optimal solution.
- **Execution:** Once a **solution is found, the actions** it recommends can be **carried out** called execution. It executes the best **optimal solution** found from the searching algorithms to reach the goal state from the current state.

Goal-based agents

- **Formulation**: goal and problem
- Given: initial state

- **Goal**: To reach the specified goal (a state) through the execution of appropriate actions.
- **Search** for a suitable action sequence and **execute the actions**

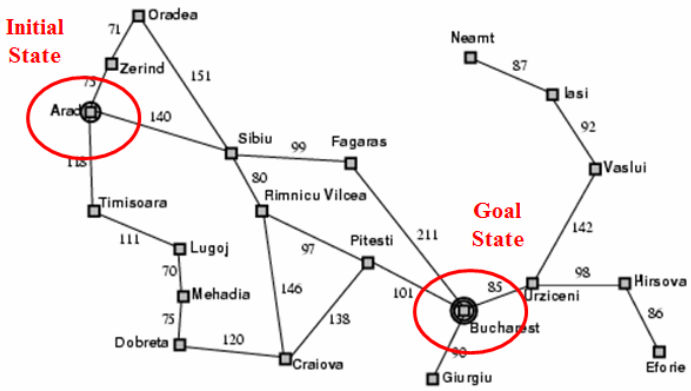
Example: Path Finding problem

Formulate goal:
- **be in Bucharest (Romania)**
-

Formulate problem:
- **action: drive between pair of connected cities (direct road)**
-
- **state: be in a city (20 world states)**

Find solution:
- **sequence of cities leading from start to goal state, e.g., Arad, Sibiu, Fagaras, Bucharest**
-

Execution
- **drive from Arad to Bucharest according to the solution**



Environment: fully observable (map), deterministic, and the agent knows effects of each action. Is this really the case?

Note: Map is somewhat of a "toy" example. Our real interest: *Exponentially large spaces*, with e.g. 10^100 or more states. Far beyond full search. Humans can often still handle those!
One of the mysteries of cognition.

Thus, we have a simple "formulate, search, execute" design for the agent, as shown in Figure 3.1.

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  persistent: seq, an action sequence, initially empty
              state, some description of the current world state
              goal, a goal, initially null
              problem, a problem formulation

  state ← UPDATE-STATE(state, percept)
  if seq is empty then
      goal ← FORMULATE-GOAL(state)
      problem ← FORMULATE-PROBLEM(state, goal)
      seq ← SEARCH(problem)
      if seq = failure then return a null action
  action ← FIRST(seq)
  seq ← REST(seq)
  return action
```

Figure 3.1 A simple problem-solving agent. It first formulates a goal and a problem, searches for a sequence of actions that would solve the problem, and then executes the actions one at a time. When this is complete, it formulates another goal and starts over.

Well-defined problems and solutions

Components involved in problem formulation/definition

1. **Initial State:** It is the starting state or initial step of the agent towards its goal.
   For example, the initial state for our agent in Romania might be described as In(Arad).

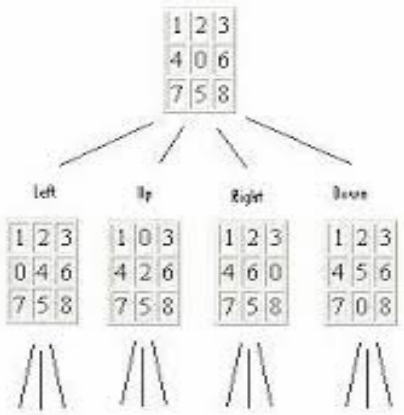
2. **Actions/ Operators:** It is the description of the possible actions available to the agent. Given a particular state s,

   ACTIONS(s) returns the set of actions that can be executed in s. We say that each of these actions is **applicable** in s.

   For example, from the state In(Arad), the applicable actions are {Go(Sibiu), Go(Timisoara), Go(Zerind)}.

3. **Transition Model/ Successor function:** It describes what each action does.
   specified by a function RESULT(s, a) that returns the state that results from doing action a in state s. We also use the term **successor** to refer to any state reachable from a given state by a single action.
   For example, we have
   RESULT(In(Arad),Go(Zerind)) = In(Zerind)

Together, the initial state, actions, and transition model implicitly define the **state space** of the problem—the set of all states reachable from the initial state by any sequence of actions.
The state space **forms** a directed network or **graph** in which the nodes are states and the links between nodes are actions.



A **path** in the state space is a sequence of states connected by a sequence of actions.

4. **Goal Test:** It determines whether a given state is a goal state. Sometimes there is an explicit set of possible goal states, and the test simply checks whether the given state is one of them.
   The agent's goal in Romania is the singleton set {In(Bucharest)}.

Sometimes the goal is specified by an abstract property rather than an explicitly enumerated set of states. For example, in chess, the goal is to reach a state called "checkmate," where the opponent's king is under attack and can't escape.

Explicit enumerated set, e.g. In(Bucharest)
Implicit abstract, e.g. checkmate

5. **Path cost:** It assigns a numeric cost to each path to achieve the goal. The problem-solving agent selects a cost function, which reflects its own performance measure.
   Step cost of taking action a in state s to reach state s' is denoted by c(s, a,s'). We assume that step costs are non-negative.

   An optimal solution has the lowest path cost among all the solutions.

   - The preceding elements (initial state, actions, transition model, goal test, path cost) define a **problem** and can be gathered into a single data structure that is given as **input to a problem-solving algorithm**.

   - A **solution** to a problem is an **action sequence** that leads from the initial state to a goal state. Solution quality is measured by the path cost function, and an **optimal solution** has the lowest path cost among all solutions.

Formulating Problems

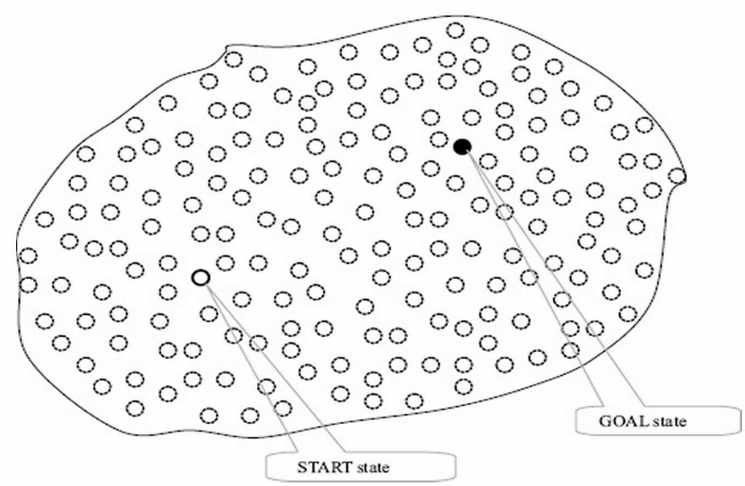
Road map

a formulation of the problem of getting to Bucharest in terms of the initial state, actions, transition model, goal test, and path cost. This formulation seems reasonable, but it is still a *model*—an abstract mathematical description.

- Real world is complex and has more details
- Irrelevant details should be removed from state space and actions, which is called abstraction.
- What's the appropriate level of abstraction?
  - the abstraction is valid, if we can expand it into a solution in the **more detailed** world
  - the abstraction is useful, if carrying out the actions in the solution is **easier than the original problem.**

State space representation

- A **state–space representation** allows for the formal definition of a problem, which makes the movement from initial state to the goal state quite easily. So, we can say that various problems like planning, learning, theorem proving, etc., are all essentially search problems only.



- **State–space search** is a process used in the field of computer science, including AI, in which successive configurations or states of an instance are considered, with the goal of finding a goal state with a desired property.

Fig: The state space

